

PHISHAWARE

PHISHING AWARENESS SIMULATION

Name	Radifa Khanam
Project	Phishing Awareness Simulation
Project Status	Implemented and tested
Environment	Kali Linux
Language	Python 3.14.7
Framework	Flask 3.1.3
Frontend	HTML + CSS
Data Storage	Local CSV

Project Report

1. Executive Summary

PhishAware is a Python and Flask based local simulation that presents three simulated email scenarios and asks the user to classify each message as phishing or legitimate. The application provides immediate feedback, highlights suspicious indicators, records choices locally, and summarizes the captured responses on a dashboard.

The implementation is self-contained and does not send real emails or request, store, or transmit passwords, authentication codes, or other sensitive credentials. The supplied test evidence shows 11 attempts, 8 correct identifications, and a displayed score of 73% for the recorded test state.

2. Project Overview

The project demonstrates an end-to-end workflow for reviewing suspicious messages: scenario selection, email inspection, classification, result feedback, guidance, and dashboard measurement. Flask manages the routes and templates, while a local CSV file stores scenario IDs and classification choices.

3. Problem Statement

Suspicious messages may use urgency, account warnings, unexpected document requests, password notices, or other persuasive patterns. A controlled local exercise can demonstrate these cues without contacting real recipients or collecting real credentials. PhishAware addresses this need through predefined scenarios, classification choices, explanatory feedback, and local measurement.

4. Objectives and Scope

- Create a controlled local simulation with multiple suspicious-email scenarios.
- Allow review of sender, subject, message body, urgency, and context.
- Record classification choices without collecting credentials.
- Provide immediate explanations of suspicious indicators.
- Display practical guidance for handling suspicious messages.
- Summarize responses through a dashboard.

Scope: Local demonstration and awareness evaluation only. No real messages are delivered and no passwords are collected.

5. Core Features

- Three predefined simulated email scenarios.
- Sender, subject, message-body, and context review.
- Phishing / legitimate classification controls.
- Immediate result page with indicators and safe-handling guidance.
- Local CSV response recording.
- Dashboard showing attempts, correct identifications, and score.

6. Evidence Snapshot

Evidence	Observed output
Main page	Three scenarios and dashboard navigation.
Scenario pages	Email details and classification controls.
Result page	Assessment, indicators, and guidance.
Dashboard	11 attempts, 8 correct identifications, 73% displayed score.

7. Requirements Analysis

Requirement	Implementation
Environment	Kali Linux
Language	Python 3.14.7
Framework	Flask 3.1.3
Frontend	HTML + CSS
Storage	Local CSV
Dashboard	Calculated response metrics

8. Tools and Technologies

Technology	Role
Python 3.14.7	Application logic and file handling
Flask 3.1.3	Local server, routes, and template rendering
HTML	Page structure and content
CSS	Page layout and visual presentation
CSV	Local response persistence
Kali Linux	Development and testing environment

9. System Architecture

The browser communicates with the local Flask application. Flask stores scenario definitions, renders templates, receives classification choices, appends responses to the CSV file, and calculates dashboard statistics.

Layer	Flow
User interface	Browser → Scenario selection → Email review
Application	Flask routes → Result processing → Dashboard
Data	Scenario ID + classification → responses.csv

10. Application Workflow

Step	Action
1. Open	Launch the local PhishAware home page.
2. Select	Choose one of three simulated email scenarios.
3. Review	Inspect sender, subject, body, urgency, and context.
4. Classify	Choose phishing or legitimate.
5. Result	View assessment and suspicious indicators.
6. Guidance	Read practical safe-handling guidance.
7. Dashboard	View attempts, correct identifications, and score.

11. Functional Modules

Module	Purpose
Scenario	Stores and presents scenarios.
Email Review	Displays email details and choices.
Result	Provides assessment and indicators.
Guidance	Displays safe-handling recommendations.
Response	Appends scenario ID and choice to CSV.
Dashboard	Calculates attempts, correct identifications, and score.

12. Implementation Details

- Scenario definitions contain an ID, sender, subject, body, and suspicious-indicator list.
- Flask routes connect the home page, scenario pages, result processing, and dashboard.
- Only scenario ID and classification are stored in data/responses.csv.
- Dashboard metrics are calculated from the recorded response state.

13. Simulation Scenarios

No.	Subject	Indicators
1	Urgent: Your Account Requires Verification	Urgency; unexpected warning; sender verification
2	Important: Employee Document Review	Unexpected document; sender verification; attachment caution
3	Your Password Will Expire Soon	Password urgency; unexpected notice; official-channel verification

14. Response Tracking and Dashboard

Each submitted classification is appended to the CSV response file. The dashboard reads the stored rows and calculates attempts, correct identifications, and the displayed percentage. The supplied test screenshot records 11 attempts, 8 correct identifications, and 73%.

15. Testing and Results

Test	Expected	Result
Startup	Local page loads	Pass
Navigation	All scenarios open	Pass
Classification	Result page loads	Pass
Feedback	Indicators and guidance appear	Pass
CSV storage	Choice is appended	Pass
Dashboard	Metrics appear	Pass
Credential handling	No credentials collected	Pass by design

16. Security and Privacy Considerations

- Runs locally on 127.0.0.1 and does not send simulated emails externally.
- Does not request, store, or transmit passwords or authentication codes.
- Stores only scenario IDs and classification choices in the local CSV.
- Uses placeholder sender addresses rather than real accounts.
- Focuses on recognition and response evaluation rather than credential capture.

17. Evidence Screenshots

The screenshots below document the implemented interface and the main user interaction stages.

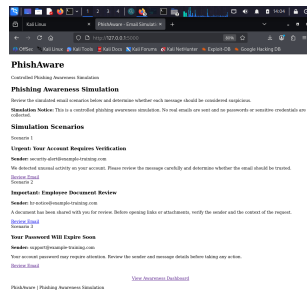


Figure 1 — Main page

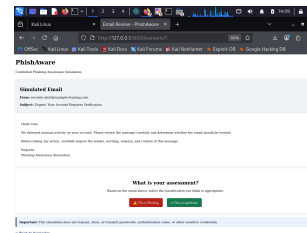


Figure 2 — Scenario 1

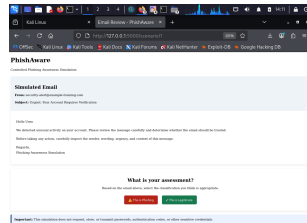


Figure 3 — Scenario 2

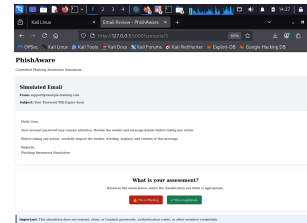


Figure 4 — Scenario 3

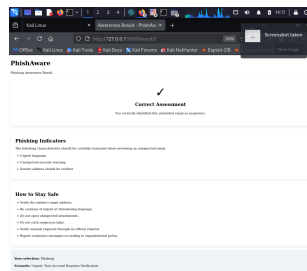


Figure 5 — Awareness result

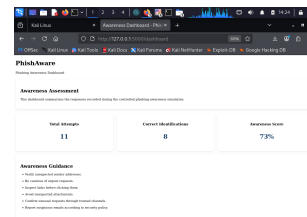


Figure 6 — Awareness dashboard

Evidence Interpretation

The screenshots demonstrate the complete local interaction path from scenario selection through message review, classification, feedback, and dashboard measurement. The dashboard evidence shows the captured test state described in this report.

18. Limitations

- Predefined scenarios rather than dynamically generated messages.
- Local CSV storage is not intended for multi-user deployment.
- Dashboard provides simple aggregates rather than historical analytics.
- No external mail-platform integration.
- Captured score is a project test state, not a statistically representative measurement.

19. Future Enhancements

- Add more scenario categories and randomized ordering.
- Add scenario difficulty levels.
- Add dashboard charts and anonymized result export.
- Improve accessibility and responsive layout.
- Add a controlled scenario-management interface.

20. Conclusion

PhishAware provides a complete local workflow for phishing-awareness simulation: scenario presentation, message review, classification, immediate feedback, local response storage, and dashboard-based measurement. The project demonstrates how a small Flask application can turn common suspicious-email patterns into an interactive and measurable exercise without requiring real email delivery or sensitive credential collection.

The supplied evidence shows that the application presents three distinct scenarios, accepts classification choices, provides explanatory suspicious-message indicators and practical guidance, and summarizes captured responses in a dashboard. The project therefore meets its core functional goal of providing a controlled environment for examining suspicious-email indicators and user responses while keeping the interaction local and privacy-conscious.

The implementation also provides a clear foundation for future expansion. Additional scenarios, richer dashboard analytics, accessibility improvements, randomized exercises, and controlled scenario management could be added without changing the basic architecture.

21. References

1. Flask Documentation — Quickstart — <https://flask.palletsprojects.com/en/stable/quickstart/>
2. Flask Documentation — Installation — <https://flask.palletsprojects.com/en/stable/installation/>
3. Flask Documentation — Web Security — <https://flask.palletsprojects.com/en/stable/web-security/>
4. Flask Documentation — Tutorial — <https://flask.palletsprojects.com/en/stable/tutorial/>
5. Flask API Documentation — <https://flask.palletsprojects.com/en/stable/api/>
6. Python Documentation — <https://docs.python.org/3/>
7. Python CSV Module — <https://docs.python.org/3/library/csv.html>
8. Python venv Documentation — <https://docs.python.org/3/library/venv.html>
9. MDN Web Docs — HTML — <https://developer.mozilla.org/en-US/docs/Web/HTML>
10. MDN Web Docs — Basic HTML Syntax — https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/Basic_HTML_syntax
11. MDN Web Docs — CSS — <https://developer.mozilla.org/en-US/docs/Web/CSS>
12. MDN Web Docs — CSS Styling Basics — https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics
13. MDN Web Docs — CSS Box Model — https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Box_model
14. OWASP Documentation Library — <https://docs.owasp.org/en/latest/>
15. OWASP — Phishing — <https://owasp.org/www-community/attacks/Phishing>

End of Report